Personal Reflection - James Huang

Throughout the project, I was responsible for the maximal-matching 2-approximation algorithm for the minimum vertex cover problem. I contributed by overseeing AI-assisted development and implementation, personally writing and formatting report sections, requesting additional relabelling and benchmark consistency tests, rewriting my presentation script, and creating the worked-example slides. I used ChatGPT and Codex for explanations, implementation and test design, and feedback on the technical accuracy of my report, script, and slides. This support required me to review the outputs and develop my understanding rather than assume that functioning code was sufficient.

My understanding developed through questions about the problem and its representation. Initially, I needed clarification that a vertex cover selects vertices touching every edge, rather than selecting edges themselves. I also questioned whether straight-line examples meant the implementation only supported paths. Learning that connections define the graph, independently of its drawing, helped me interpret other graph families and construct presentation examples.

The four-vertex path clarified the difference between an approximation and an exact solution. My algorithm returns four vertices even though two are sufficient. Understanding why this remains acceptable helped me explain the factor-two guarantee accurately: the returned cover contains at most twice the optimum number of vertices. It does not promise an optimal answer or only two additional vertices.

The most challenging part was explaining why the implementation preserves this guarantee. While rewriting my script, I spent the most time on the two invariants of my algorithm: selected matching edges have disjoint endpoints, and every processed edge has a marked endpoint. These connected the code to the proof. Disjoint edges require distinct covering vertices, establishing a lower bound, while permanent marks ensure that processed edges remain covered.

Through creating an incorrect worked example, I strengthened this understanding. For the path 0–1–2, incorrectly accepting both edges produces overlapping selected edges. Their count is two, although vertex 1 alone covers the graph, so the claimed lower bound fails. I created a slide highlighting the shared endpoint and refined its labels to distinguish the incorrect modification from the actual algorithm. This made the condition meaningful rather than something I simply repeated.

Testing introduced another learning experience. I initially associated relabelling with preserving the same behaviour and solution. I learned that the optimum remains unchanged, but sorted traversal can produce a different matching and approximation size. This informed my explanation of the additional tests. Benchmark consistency checks also showed me that correct computation and accurate recording are separate concerns.

Next time, I would trace examples and record invariants before requesting implementation assistance. I would keep notes about misunderstandings, corrections, and decisions throughout development, and establish expected test outcomes beforehand. Throughout the project, I learned to assess an algorithm through its assumptions, guarantees, and evidence.

Rewriting explanations and building diagrams helped me move from recognising its steps to explaining why those steps are necessary and what would fail if they changed.